M2PGi – M2M

Pr. Olivier Gruber (olivier.gruber@imag.fr)

Laboratoire d'Informatique de Grenoble

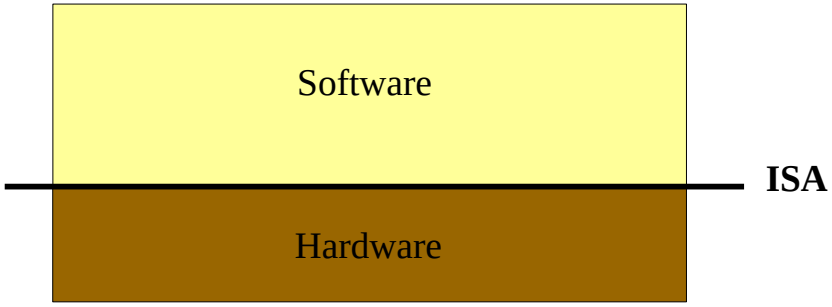
Université de Grenoble-Alpes

© Pr. Olivier Gruber

Today

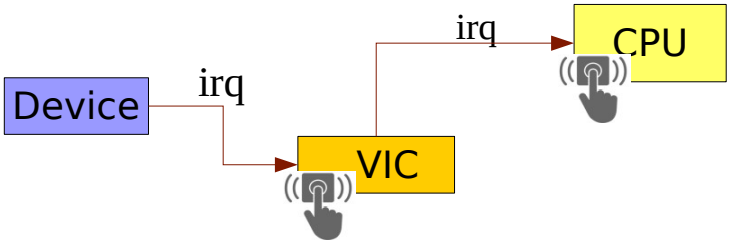
- Hardware Support for Interrupts
  - Halt instruction, Processor modes, Interrupt Requests, Exception vectors, etc.
- Software Support for Interrupts
  - Handling interrupts
  - ***Race condition challenge***

    & Solutions (compatible with an event-oriented execution)



© Pr. Olivier Gruber

Summary



The instruction "halt" such as ***WFI*** or ***WFE*** for ARM processors

Vectored Interrupt Controller: (architected state per interrupt)

Trapping for the Interrupt Processing by the processor

FIQ	SUBS PC, R14_fiq, #4	PC + 4	PC+4	Where the PC is the address of the instruction that was not executed because the FIQ or IRQ took priority
IRQ	SUBS PC, R14_irq, #4	PC + 4	PC+4	

	Privileged modes					
		Exception modes				
User	System	Supervisor	Abort	Undefined	Interrupt	Fast interrupt
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12_fiq
R13	R13	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
R14	R14	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
PC	PC	PC	PC	PC	PC	PC
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

indicates that the normal register used by User or System mode has been replaced by an alternative register specific to the exception mode

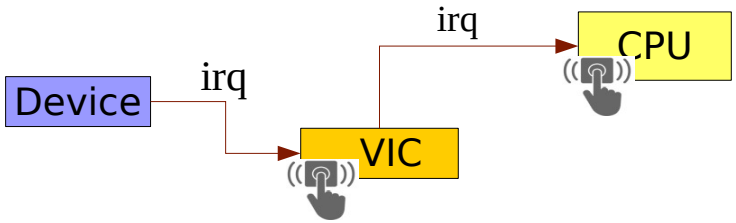
```
0x00    br    _reset
0x04    br    _undef_inst
0x08    br    _soft_irq
0x0c    br    _prefetch_abort
0x10    br    _data_abort
0x14    br    _not_used
0x18    br    _isr
0x1c    br    _fisr
```

```
_isr:
    sub     lr,lr,#4
    stmfd   sp!, {r0-r12, lr}
    bl isr  // C function call
    ldmfd   sp!, {r0-r12, pc}^
```

The C function "isr()" is the real processing of the interrupt requests, attending the requesting devices

© Pr. Olivier Gruber

Interrupt Service Routine – “Who is it?“




```
.global _isr
_isr:
    ...
    ; now call the interrupt_service_routine
    bl isr
    ...
```

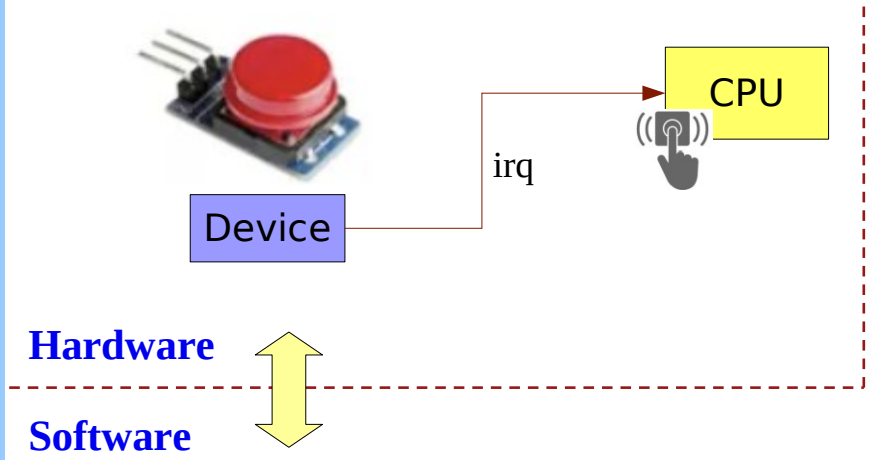
```
struct handler {
  void (*callback)(void*);
  void* cookie;
};

struct handler handlers[MAX_NHANDLERS];

void isr() {
  uint32_t irqs = vic_load_irqs();
  for (uint32_t i=0 ; i<32 ; i++) {
    struct handler_t* handler;
    handler = &handlers[i];
    if (irqs & (1<<i))
      handler->callback(handler->cookie);
  }
  vic_ack_irqs(irqs);
  return;
}
```

```
void vic_enable_interrupt(uint32_t irqno,
                          void (*handler)(void*), void* cookie);
void vic_disable_interrupt(uint32_t irqno);
```

```
uint8_t button_get_status(void* bar);

void button_enable_interrupt(void (*handler)(void*),
                             void* cookie) {
  vic_enable_interrupt(BUTTON_IRQ,handler,cookie);
  ... // enable interrupt at the button device
}

void button_disable_interrupt() {
  ... // disable interrupt at the button device
  vic_disable_interrupt(BUTTON_IRQ);
}
```

© Pr. Olivier Gruber

Button Interrupt – An Acceptable Short Handler




```
uint8_t button_get_status(void* bar);

void button_enable_interrupt(void (*handler)(void*),
                             void* cookie);
void button_disable_interrupt();
```

```
void core_enable_interrupts();
void core_disable_interrupts();

void _start() {
  bl_init();
  core_enable_interrupts();
  for (;;) {
    halt();
  }
}
```

enable the core to be interrupted before forcing it to sleep...

Handler for the button interrupt

```
static
void bl_handler(void* cookie) {
  uint8_t status;
  status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
}

void bl_init() {
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);

  button_enable_interrupt(bl_handler,
                          null);
}
```

enable the button interrupt and associate the given handler

© Pr. Olivier Gruber

UART Interrupt – Unacceptable Handler


```
void core_enable_interrupts();
void core_disable_interrupts();


void _start() {
  shell_init();
  core_enable_interrupts();
  for (;;) {
    halt();
  }
}
```

```
uint8_t uart_receive(void* uart);
void    uart_send(void* uart, uint8_t byte);


void uart_enable_interrupt(void (*handler)(void*),
                           void* cookie);

void uart_disable_interrupt();
```

Not an acceptable handler

Remember: interrupts are disabled while executing a handler

Remember: uart_send may block...

Remember: uarts have small FIFO queues...

```
char line[80];
int offset = 0;

static
void _uart_handler(void* cookie) {

  uint8_t code = uart_receive(UART0);
  while (code != 0) {
    uart_send(UART0, code);
    if (code == '\n') {
      shell(line, offset);
      offset=0;
    } else
      line[offset++]=(char)code;

    code = uart_receive(UART0);
  }
}


void shell_init() {

  uart_init(UART0);

  uart_enable_interrupt(_uart_handler,
                        null);
}
```

© Pr. Olivier Gruber

Another Design for the UART0


```
void core_enable_interrupts();
void core_disable_interrupts();

extern char line[80];
extern int length;
extern int offset;

void _start() {
  shell_init();
  core_enable_interrupts();
  int last = 0;
  for (;;) {
    while (last < offset)
      uart_send(UART0, line[last++]);

    if (length != 0) {
      shell(line, length);
      length = last = 0;
    }
    halt();
  }
}
```

Using a shared buffer for a line.

What do we think?

```
char line[80];
int offset = 0;
int length = 0;

static
void _uart_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code != 0) {
    if (code == '\n') {
      length = offset;
      offset=0;
    } else
      line[offset++]=(char)code;
    code = uart_receive(UART0);
  }
}
```

© Pr. Olivier Gruber

Another Design for the UART0 – **Flawed!**


```c
void core_enable_interrupts();
void core_disable_interrupts();

extern char line[80];
extern int length;
extern int offset;

void _start() {
  shell_init();
  core_enable_interrupts();
  int last = 0;
  for (;;) {
    while (last < offset)
      uart_send(UART0, line[last++]);

    if (length != 0) {
      shell(line, length);
      length = last = 0;
    }
    halt();
  }
}
```

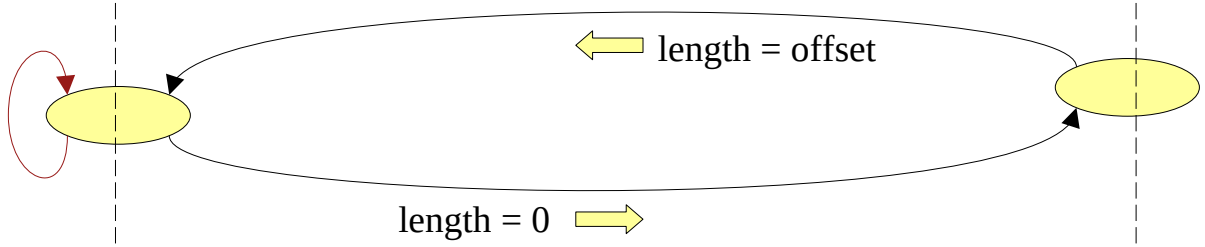
DOES NOT WORK!

```c
char line[80];
int offset = 0;
int length = 0;

static
void _uart_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code != 0) {
    if (code == '\n') {
      length = offset;
      offset=0;
    } else
      line[offset++]=(char)code;
    code = uart_receive(UART0);
  }
}
```

buffer is owned by the main loop world

Wrong ownership management of the line buffer

buffer is owned by the handler world...



© Pr. Olivier Gruber

Yet Another Design for the UART0...


```
void core_enable_interrupts();
void core_disable_interrupts();

char line[80];
int offset;
extern list_t list;
void _start() {
  shell_init();
  core_enable_interrupts();
  int last = 0;
  for (;;) {
    while (!list_empty(&list)) {
      uint8_t code = list_remove(&list,0);
      uart_send(UART0, code);
      if (code == '\n') {
        shell(line, offset);
        offset = 0;
      } else
        line[offset++]=(char)code;
    }
    halt();
  }
}
```

Let's try using a list:

- the handler appends at the end…
- the main loop removes at the beginning...

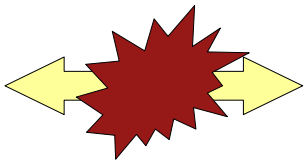
What do we think?

```
list_t list; // list of bytes

static
void _uart_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code != 0) {
    list_append(&list,code);
    code = uart_receive(UART0);
  }
}
```

© Pr. Olivier Gruber

Yet Another Design for the UART0… Also **Flawed!**


```
void core_enable_interrupts();
void core_disable_interrupts();

char line[80];
int offset;
extern list_t list;
void _start() {
  shell_init();
  core_enable_interrupts();
  int last = 0;
  for (;;) {
    while (!list_empty(&list)) {
      uint8_t code = list_remove(&list,0);
      uart_send(UART0, code);
      if (code == '\n') {
        shell(line, offset);
        offset = 0;
      } else
        line[offset++]=(char)code;
    }
    halt();
  }
}
```



Race Conditions

```
list_t list; // list of bytes

static
void _uart_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code != 0) {
    list_append(&list,code);
    code = uart_receive(UART0);
  }
}
```

The list is shared and manipulated concurrently…

Using synchronization tools (like semaphores or mutexes) are not possible: blocking the handler would mean a deadlock since the handler is blocking the main loop...

© Pr. Olivier Gruber

Yet Another Design for the UART0… Fixed?


```
void core_enable_interrupts();
void core_disable_interrupts();

char line[80];
int offset;
extern list_t list;
void _start() {
  shell_init();
  int last = 0;
  for (;;) {
    core_disable_interrupts();
    while (!list_empty(&list)) {
      uint8_t code = list_remove(&list,0);

      core_enable_interrupts();
      uart_send(UART0, code);
      if (code == '\n') {
        shell(line, offset);
        offset = 0;
      } else
        line[offset++]=(char)code;
      core_disable_interrupts();
    }
    halt();
  }
}
```

Disabling the interrupts is a way to avoid
the harmful race condition around the test-and-set pattern.

Nota Bene: the interrupts **must** be kept disabled
once the list has been found empty…
otherwise, some characters could be missed,
appended to the list just before the processor halts.

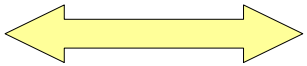
```
list_t list; // list of bytes

static
void _uart_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code != 0) {
    list_append(&list,code);
    code = uart_receive(UART0);
  }
}
```

© Pr. Olivier Gruber

Yet Another Design for the UART0… Fixed?


```
void core_enable_interrupts();
void core_disable_interrupts();

char line[80];
int offset;
extern list_t list;
void _start() {
  shell_init();
  int last = 0;
  for (;;) {
    core_disable_interrupts();
    while (!list_empty(&list)) {
      uint8_t code = list_remove(&list,0);

      core_enable_interrupts();
      uart_send(UART0, code);
      if (code == '\n') {
        shell(line, offset);
        offset = 0;
      } else
        line[offset++]=(char)code;
      core_disable_interrupts();
    }
    halt();
  }
}
```

Fixed though?

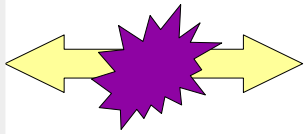


```
list_t list; // list of bytes

static
void _uart_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code != 0) {
    list_append(&list,code);
    code = uart_receive(UART0);
  }
}
```

© Pr. Olivier Gruber

Another Example – A Shell on UART0


```
void core_enable_interrupts();
void core_disable_interrupts();

char line[80];
int offset;
extern list_t list;
void _start() {
  shell_init();
  int last = 0;
  for (;;) {
    core_disable_interrupts();
    while (!list_empty(&list)) {
      uint8_t code = list_remove(&list,0);

      core_enable_interrupts();
      uart_send(UART0, code);
      if (code == '\n') {
        shell(line, offset);
        offset = 0;
      } else
        line[offset++]=(char)code;
      core_disable_interrupts();
    }
    halt();
  }
}
```



Memory Management

```
list_t list; // list of bytes

static
void _uart_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code != 0) {
    list_append(&list,code);
    code = uart_receive(UART0);
  }
}
```

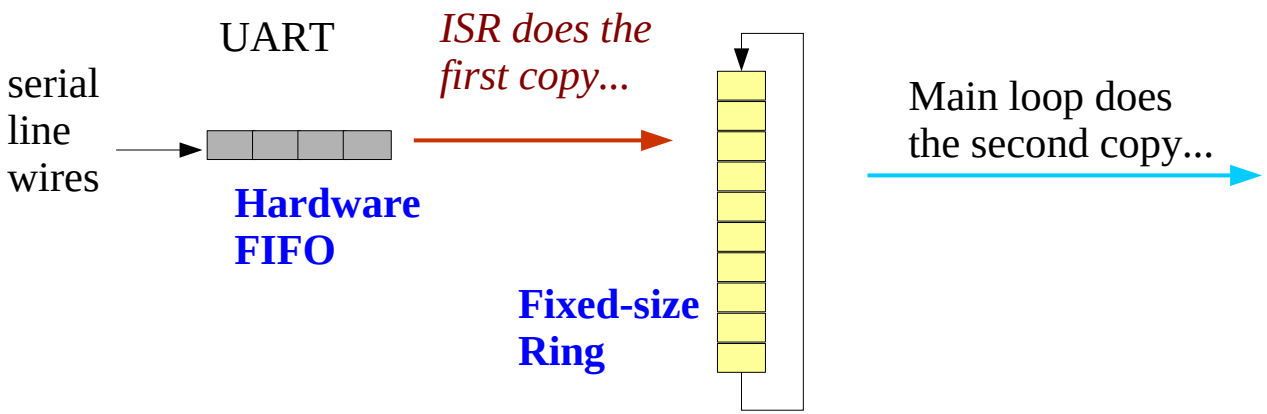
A list used that way requires dynamic memory management… **Would not work with a shared malloc/free (again, race conditions).**

So we must use a dedicated memory management solely for that list and only that list. Race conditions avoided via disabling the interrupts.

Not an efficient design/implementation. Not a good idea to allocate memory in an interrupt handler...

© Pr. Olivier Gruber

The Right Design – Fixed-size Lock-free Rings



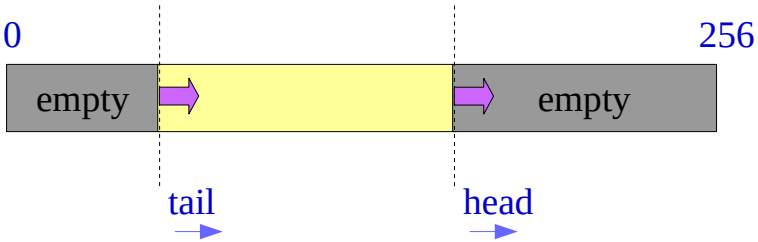
The idea is the right one:

- fixed-size ring, so no dynamic memory management
- can be implemented in way that it can be used concurrently but lock-free

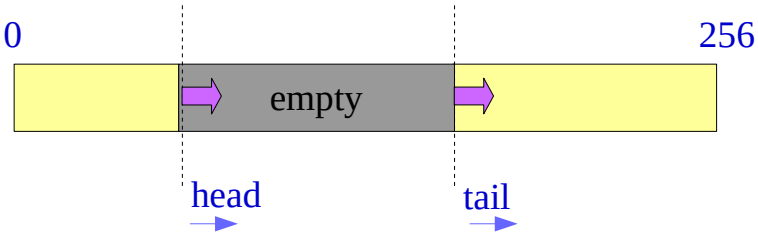
© Pr. Olivier Gruber

Lock-free Circular Buffer – Lock-Free Ring


```c
#define MAX_CHARS 512

volatile uint32_t tail = 0;
volatile uint8_t buffer[MAX_CHARS];
volatile uint32_t head = 0;

bool_t ring_empty() {
  return (head==tail);
}

bool_t ring_full() {
  int next = (head + 1) % MAX_CHARS;
  return (next==tail);
}

void ring_put(uint8_t bits) {
  uint32_t next = (head + 1) % MAX_CHARS;
  buffer[head] = code;
  head = next;
}

uint8_t ring_get() {
  uint8_t bits;
  uint32_t next = (tail + 1) % MAX_CHARS;
  bits = buffer[tail];
  tail = next;
  return bits;
}
```



Circular Buffer:
Acts as a communication channel between the ISR and the main loop



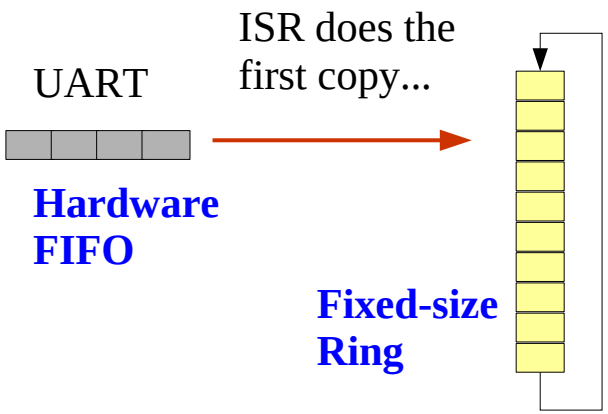
Written this way... a ring is lock-free with one producer and one consumer, even on multi-core machines, with globally ordered writes to memory

© Pr. Olivier Gruber

Coding with Fixed-Sized Lock-free Rings


```
ring_t ring;

void uart_rx_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code) {
    ring_put(&ring,code);
    code = uart_receive(UART0);
  }
}

void _start() {
  ring_init(&ring);
  uart_init(UART0, uart_rx_handler);
  core_enable_interrupts();
  for (;;) {
    process_ring();
    halt();
  }
}
char line[80];
uint8_t offset;

void process_ring() {

  while (!ring_empty(&ring)) {

    uint8_t code = ring_get(&ring);

    uart_send(UART0, code);

    if (code == '\n') {
      shell(line, offset);
      offset = 0;
    } else
      line[offset++]=(char)code;
    }
  }
}
```

Put bytes into the ring...



Get bytes from the ring...

Copy bytes into the line buffer (solely owned by the main loop world)

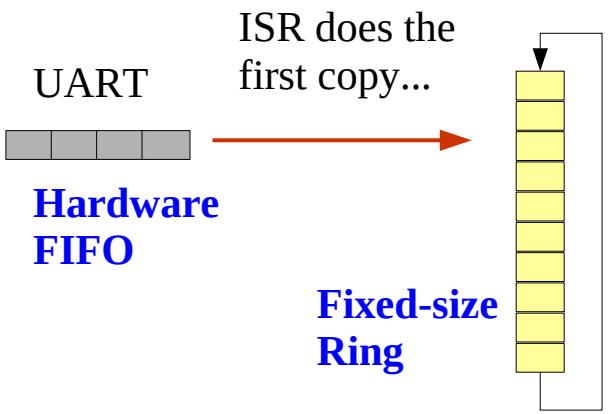
Most of the work is back on the main loop! Race conditions on the ring are OK!

© Pr. Olivier Gruber

Coding with Fixed-Sized Lock-free Rings


```
ring_t ring;

void uart_rx_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code) {
    ring_put(&ring,code);
    code = uart_receive(UART0);
  }
}

void _start() {
  ring_init(&ring);
  uart_init(UART0, uart_rx_handler);
  core_enable_interrupts();
  for (;;) {
    process_ring();
    halt();
  }
}
char line[80];
uint8_t offset;

void process_ring() {

  while (!ring_empty(&ring)) {

    uint8_t code = ring_get(&ring);

    uart_send(UART0, code);

    if (code == '\n') {
      shell(line, offset);
      offset = 0;
    } else
      line[offset++]=(char)code;
    }
  }
}
```

Put bytes into the ring...



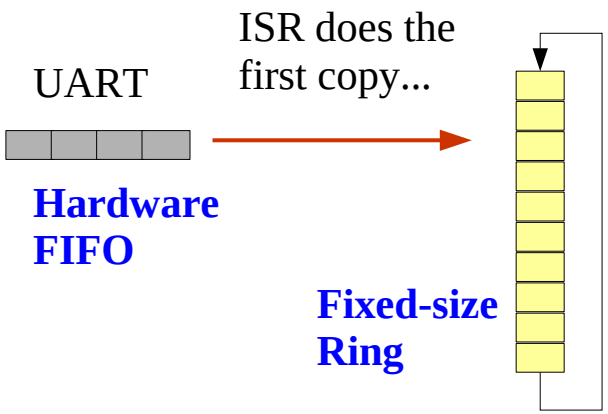
But is this code correct?
Don't we risk to not process the last characters received by the UART?

© Pr. Olivier Gruber

Coding with Fixed-Sized Lock-free Rings


```
ring_t ring;

void uart_rx_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code) {
    ring_put(&ring,code);
    code = uart_receive(UART0);
  }
}

void _start() {
  ring_init(&ring);
  uart_init(UART0, uart_rx_handler);
  core_enable_interrupts();
  for (;;) {
    process_ring();
    halt();
  }
}
char line[80];
uint8_t offset;

void process_ring() {

  while (!ring_empty(&ring)) {

    uint8_t code = ring_get(&ring);

    uart_send(UART0, code);

    if (code == '\n') {
      shell(line, offset);
      offset = 0;
    } else
      line[offset++]=(char)code;
    }
  }
}
```

Put bytes into the ring...



Not correct, there is a risk to miss the last characters

Lock-free does not mean safe **test-and-set** *patterns...*

© Pr. Olivier Gruber

Right Design – Correct Code


```
extern ring_t ring;

void _start() {
  ring_init(&ring);
  uart_init(UART0, uart_rx_handler);
  core_enable_interrupts();
  for (;;) {
    process_ring();
    core_disable_interrupts();
    if (ring_empty(&ring))
      halt();
    else
      core_enable_interrupts();
  }
}
char line[80];
uint8_t offset;

void process_ring() {
  while (!ring_empty(&ring)) {
    uint8_t code = ring_get(&ring);
    uart_send(UART0, code);
    if (code == '\n') {
      shell(line, offset);
      offset = 0;
    } else
      line[offset++]=(char)code;
    }
  }
}
```

```
ring_t ring;

void uart_rx_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code) {
    ring_put(&ring,code);
    code = uart_receive(UART0);
  }
}
```

The processor only sleeps if the ring is empty, with a test-and-set pattern that is safely protected.

Fortunately, ***when the processor goes to sleep it atomically enables the interrupts***.

And we still have:

- Most of the work is back on the main loop!
- Interrupts are enabled during that time.
- The ring is lock-free.
- No dynamic memory management

© Pr. Olivier Gruber

Right Design – Correct Code


```
extern ring_t ring;

void _start() {
  ring_init(&ring);
  uart_init(UART0, uart_rx_handler);
  core_enable_interrupts();
  for (;;) {
    process_ring();
    core_disable_interrupts();
    if (ring_empty(&ring))
      halt();
    else
      core_enable_interrupts();
  }
}
char line[80];
uint8_t offset;

void process_ring() {
  while (!ring_empty(&ring)) {
    uint8_t code = ring_get(&ring);
    uart_send(UART0, code);
    if (code == '\n') {
      shell(line, offset);
      offset = 0;
    } else
      line[offset++]=(char)code;
    }
  }
}
```

```
ring_t ring;

void uart_rx_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code) {
    ring_put(&ring,code);
    code = uart_receive(UART0);
  }
}
```

So, what do you think?

Are we finally done?

© Pr. Olivier Gruber

Right Design – Correct Code – Indeed !


```
extern ring_t ring;

void _start() {
  ring_init(&ring);
  uart_init(UART0, uart_rx_handler);
  core_enable_interrupts();
  for (;;) {
    process_ring();
    core_disable_interrupts();
    if (ring_empty(&ring))
      halt();
    else
      core_enable_interrupts();
  }
}
char line[80];
uint8_t offset;

void process_ring() {
  while (!ring_empty(&ring)) {
    uint8_t code = ring_get(&ring);
    uart_send(UART0, code);
    if (code == '\n') {
      shell(line, offset);
      offset = 0;
    } else
      line[offset++]=(char)code;
    }
  }
}
```

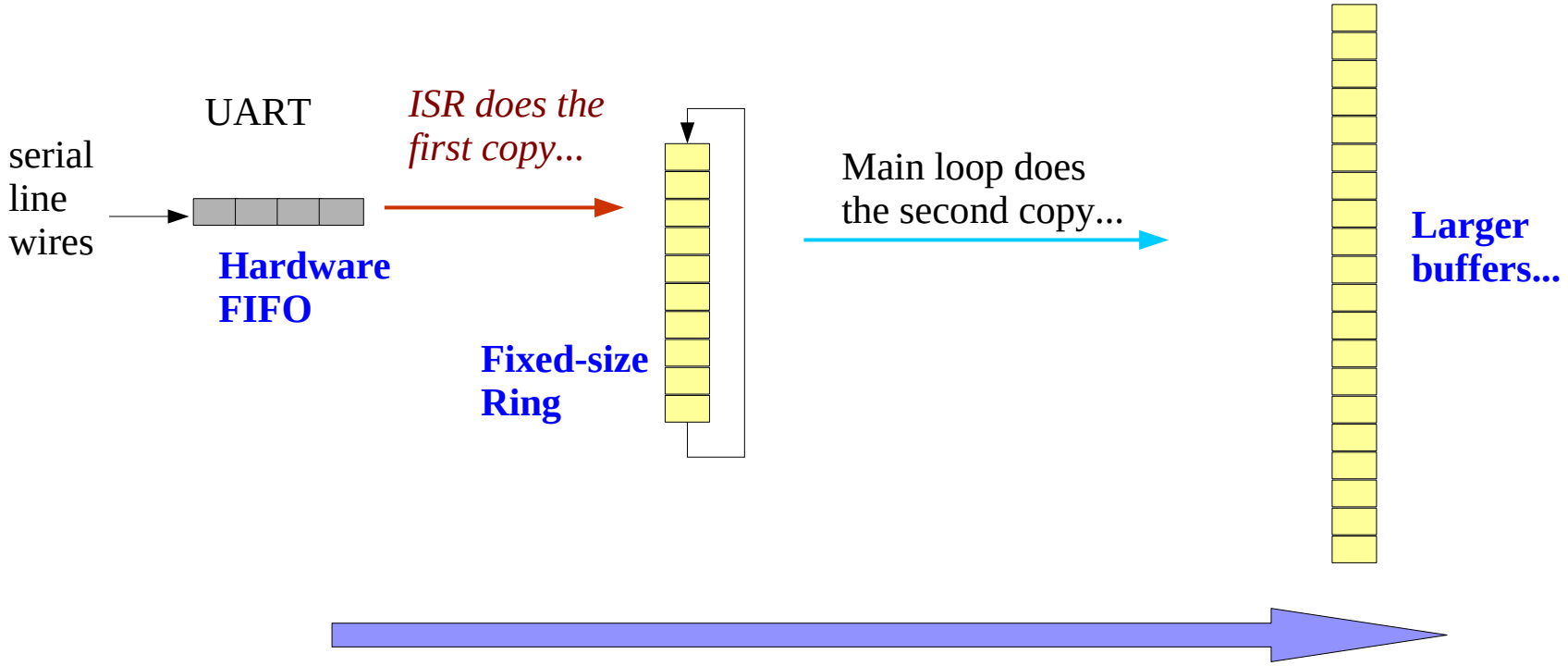
```
ring_t ring;

void uart_rx_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code) {
    ring_put(&ring,code);
    code = uart_receive(UART0);
  }
}
```

Not Bad at all !!!

Still a minor risk of a buffer overflow if the ring fills up... But there is nothing we can do about it...

© Pr. Olivier Gruber

Gaining Time Through Buffering



Buy more and more time...

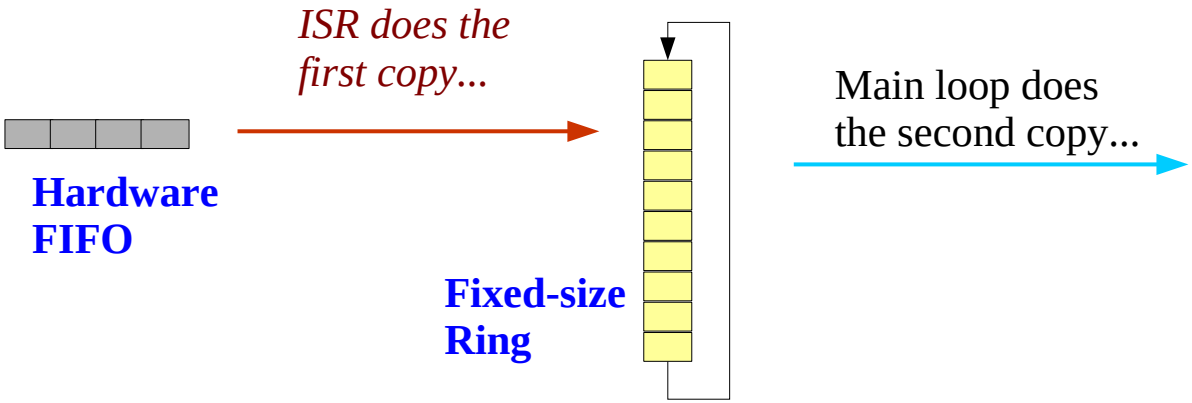
With buffers of growing sizes...

But, if using fixed-sized buffers,
there is always the risk of ***overflow***...

With dynamically allocated buffers... no risk of buffer overflow...
but there is the risk to run out of memory....

© Pr. Olivier Gruber

Three Worlds Cooperating around our special Ring




```
void _start() {
  ring_init(&ring);
  uart_init(UART0,
            uart_rx_handler);
  core_enable_interrupts();
  for (;;) {
    process_ring();
    core_disable_interrupts();
    if (ring_empty(&ring))
      halt();
    else
      core_enable_interrupts();
  }
}
```

Mr. UART
(hardware)

Mr. ISR
(interrupt service routine & handlers)

```
void uart_rx_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code) {
    ring_put(code);
    code = uart_receive(UART0);
  }
  uart_interrupt_ack();
}
```

Ms. MainLoop
(application)

© Pr. Olivier Gruber

Next Challenge – Mixing Interrupts & Events


```
ring_t ring;

void uart_rx_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  while (code) {
    ring_put(&ring,code);
    code = uart_receive(UART0);
  }
}

char line[80];
uint8_t offset;

void process_ring() {

  while (!ring_empty(&ring)) {
    uint8_t code = ring_get(&ring);
    uart_send(UART0, code);
    if (code == '\n') {
      shell(line, offset);
      offset = 0;
    } else
      line[offset++]=(char)code;
    }
  }
}

void _start() {
  ring_init(&ring);
  uart_init(UART0, uart_rx_handler);
  core_enable_interrupts();
  for (;;) {
    process_ring();
    core_disable_interrupts();
    if (ring_empty(&ring))
      halt();
    else
      core_enable_interrupts();
  }
}
```

How do we reconcile these two code structures?

```
struct event {
  void* cookie;
  void (*react)(void* cookie);
  uint64_t eta; // Estimated Time of Arrival
};

void _start() {  // event pump,

  ...

  for (;;) {
    struct event* evt = event_pop();
    if (evt!=NULL)
      evt→react(evt→cookie);
    else
      sleep_until_next_event();
  }
}
```

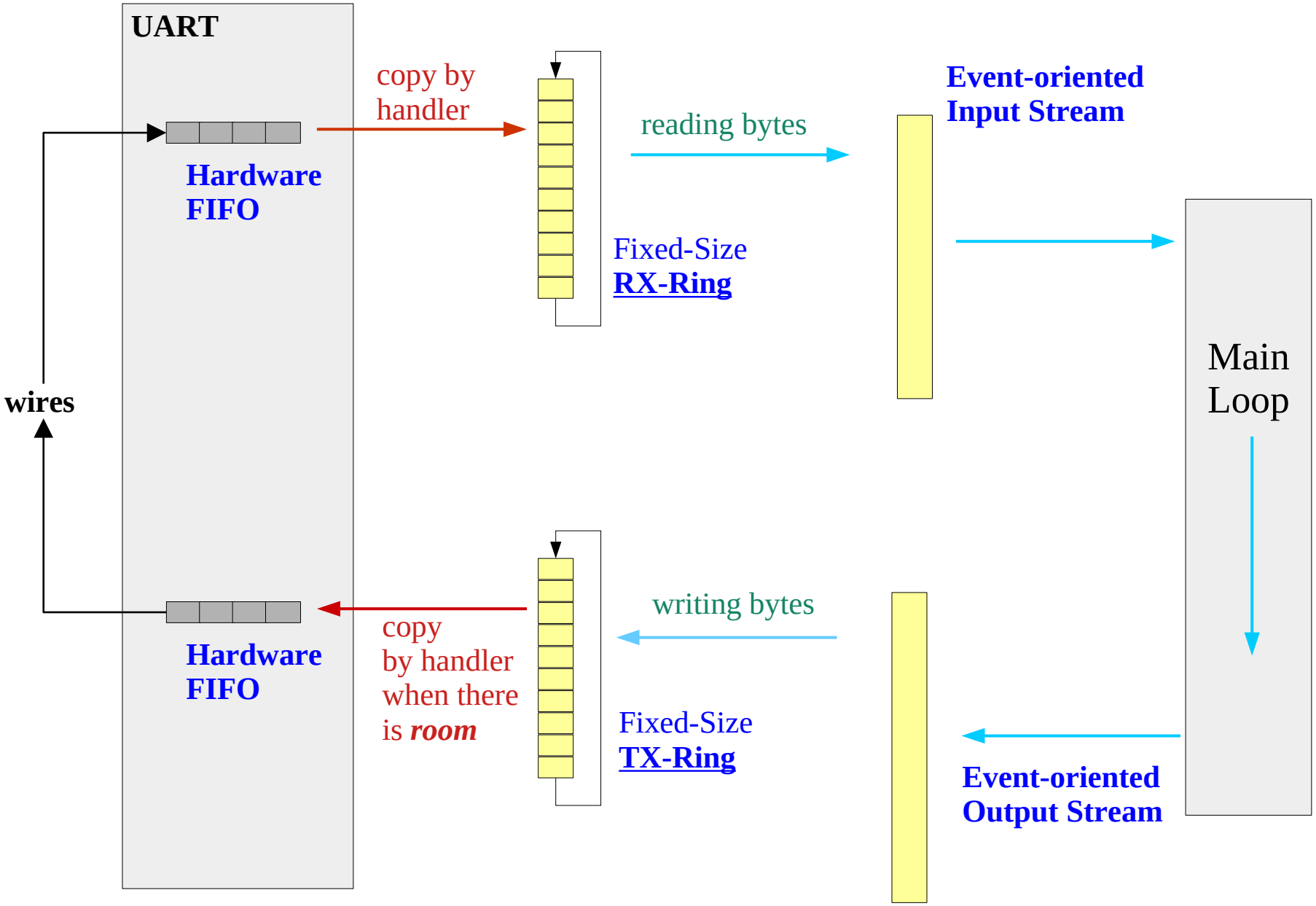
© Pr. Olivier Gruber

Mixing Interrupts & Events – Design Points

- Need a generic mechanism to service several devices
  - Need short interrupt handlers, let's call them '***top-handlers***'
    - Attend to the hardware, copy bytes into rings
  - Need event reactions, let's call them '***bottom-handlers***'
    - *To copy out bytes from rings*
  - *top-handlers need a way to request their bottom-handler counter parts*
    - *Bottom handlers must be posted in front of the queue so that they can be executed as soon as possible*
    - *Be careful, each bottom handler only needs to be posted once at any point in time*
- *Need asynchronous byte streams*
  - *To properly structure the "application code"*
  - *With asynchronous listeners*
    - *for available 'data' to read*
    - *for available 'room' to write*

© Pr. Olivier Gruber

Full Buffering Picture – Standard Streams



© Pr. Olivier Gruber

Standard Streams – Event-oriented Specification


```
/*
 * Sets the read listener for the given stream.
 * The given listener will be invoked, as part of its own event,
 * whenever there are bytes to read from the given stream, passing
 * back the given cookie.
 */
void stream_set_read_listener(int stream, void (*listener)(void*), void* cookie);

/*
 * Read up to "length" bytes in the given buffer.
 * May read zero bytes, if none are available. Never blocks.
 * Returns the number of bytes read; returns -1 if the end of stream has been reached.
 */
int stream_read(int stream, uint8_t* buffer, size_t length);

/*
 * Write as many bytes as possible, from the given buffer,
 * up to the given "length" bytes. Never blocks.
 * Returns the number of bytes written; returns -1 if the end of stream has been reached.
 */
int stream_write(int stream, uint8_t* buffer, size_t length);

/*
 * Sets the listener on the given stream to be called when
 * there is available room to write bytes.
 * The given listener will be invoked, as part of its own event,
 * passing back the given cookie.
 * Remove this listener when there is nothing to write by setting
 * NULL as listener and cookie.
 */
void stream_set_write_listener(int stream, void (*listener)(void*), void* cookie);
```

© Pr. Olivier Gruber

Conclusion

- ToDo List
  - Integrated interrupts & events
    - Timer integration for an interrupt every millisecond to provide a notion of passing time
    - Event-oriented streams above the asynch UART0 send/receive, with RX/TX interrupts
    - For debug, keep the kprintf directly above a blocking uart_send() but use the UART1 (via a socket)
  - Demonstration
    - Console shell above the “uart0 input/output stream”
      - Top line with the current passing time, % cpu usage, number of events, updated every second
      - Clear screen (ctrl-c), edited current line, left/right arrows, delete, backspace
      - Home-made blinking cursor (half-second period)
      - Command lines validated by `enter` and given through a listener
    - History via a <u>fully asynchronous</u> service
      - Listens for validated lines and keep a fixed-length log of them
      - Console decodes up/down arrows to request lines asynchronous from the history service
    - Vintage game (optional)

© Pr. Olivier Gruber